

Практическое занятие №3

https://github.com/zhilovhub/UniveristyFrameworks/tree/master/Courses_pr3

1. Цель работы

Навести порядок в настройках веб-приложения и добавить базовую защиту, чтобы служба не была открыта всему миру на равных правах. Ключевой навык — ранняя проверка настроек и понятные причины отказа, когда параметры заданы неверно.

В качестве предметной области выбран учёт лабораторных работ студентов: каждая лабораторная имеет название (**title**) и максимальный балл (**maxScore**).

2. Постановка задачи

1. Реализовать чтение настроек из трёх источников с явным порядком приоритета.
2. Реализовать раннюю проверку корректности настроек и запретить запуск при ошибках.
3. Реализовать ограничение запросов из браузера только от доверенных источников.
4. Реализовать ограничение частоты запросов, чтобы один источник не мог завалить службу.
5. Добавить минимум два защитных заголовка ответа.
6. Реализовать два режима работы — учебный и боевой, переключение режима только настройкой.

Дополнительно: разные лимиты для разных маршрутов (создание ниже, чем чтение списка), проверка корректности адресов доверенных источников, нагрузочная проверка.

3. Используемые технологии

- Node.js (встроенный модуль **node:http**)
- ES Modules
- Встроенный тестовый раннер **node --test**

Сторонние зависимости не используются — всё реализовано на стандартной библиотеке Node.js.

4. Структура проекта

```
Courses_pr3/
├── config/
│   └── appsettings.json
├── src/
│   ├── config.js
│   ├── security.js
│   ├── labs.js
│   └── server.js
├── test/
│   ├── configPriority.test.js
│   ├── securityAndLimit.test.js
│   └── loadTest.js
```

```
├─ package.json
└─ README.md
```

5. Описание реализации

5.1. Чтение настроек из трёх источников

В файле `src/config.js` реализовано чтение настроек из:

1. Файла конфигурации `config/appsettings.json` — самый низкий приоритет.
2. Переменных окружения с префиксом `APP_` (например, `APP_MODE`, `APP_PORT`, `APP_TRUSTED_ORIGINS`, `APP_READ_PER_MINUTE`, `APP_WRITE_PER_MINUTE`) — средний приоритет.
3. Аргументов командной строки в формате `--ключ=значение` (например, `--port=4000`, `--mode=боевой`) — высший приоритет.

Функция `buildConfig` склеивает три источника в указанном порядке: значения из файла кладутся в основу, далее перекрываются переменными окружения, а на самом верху — аргументы командной строки. Это позволяет в одном и том же образе менять настройки для разных сред без пересборки.

5.2. Ранняя проверка корректности

Функция `validateConfig` проверяет:

- режим работы — допустимы только значения `учебный` и `боевой`;
- порт — целое число от 1 до 65535;
- список доверенных источников — не пуст и не содержит дубликатов;
- каждый источник — корректный URL со схемой `http/https`;
- для `localhost` обязательно указан порт;
- лимиты чтения и записи — целые положительные числа, причём лимит записи не выше лимита чтения.

Если хотя бы одна проверка не прошла, сервер не стартует и печатает в `stderr` подробный список ошибок.

5.3. Базовая защита

В файле `src/security.js`:

- Защитные заголовки (`applySecurityHeaders`):
 - `X-Content-Type-Options: nosniff` — защита от MIME-sniffing;
 - `X-Frame-Options: DENY` — защита от clickjacking;
 - `Cache-Control: no-store, max-age=0` и `Pragma: no-cache`.
- CORS-проверка (`applyCORS`): разрешающие заголовки выдаются только если `Origin` входит в `trustedOrigins`. Иначе на запрос возвращается 403.
- Ограничение частоты запросов (`createRateLimiter`): для каждого IP-адреса ведутся два счётчика — на чтение и на запись. Окно — 60 секунд. Лимиты отдельные: например, 60 запросов на чтение и 20 на запись в минуту. При превышении возвращается 429 с заголовком `Retry-After: 60`.

5.4. Два режима работы

Режим переключается параметром `mode` (в файле, переменной окружения или аргументе) — без изменений в коде:

- учебный — подробные сообщения об ошибках: указывается конкретный недоверенный источник, доступное число запросов, имя несуществующего ID;
- боевой — минимальные сообщения: `Forbidden`, `Too Many Requests`, `Not Found`, `Bad Request`.

5.5. Маршруты API

Метод	Путь	Действие
GET	<code>/api/labs</code>	список всех лабораторных работ
GET	<code>/api/labs/by-id/:id</code>	получение одной работы по ID
POST	<code>/api/labs</code>	создание новой работы (<code>title</code> , <code>maxScore</code>)
GET	<code>/api/mode</code>	текущий режим и активные лимиты

6. Запуск приложения

```
npm start
npm run start:prod
npm run start:custom
node src/server.js --mode=боевой --port=4000
```

7. Тестирование

7.1. Модульные тесты

Команда `npm test` запускает встроенный раннер `node --test`. Тесты разделены на два файла:

- `test/configPriority.test.js` — приоритет источников и валидация.
- `test/securityAndLimit.test.js` — CORS и rate limiter.

Все тесты проходят успешно.

```
zhilovilya@zhilovilya-mac Courses_pr3 % npm test
```

Результаты:

```
Успешно (200): 0
Rate limited (429): 0
Ошибки: 100
Время: 0.01с
RPS: 6666.67
```

Тест 3: Запись (POST), средняя нагрузка

```
URL: http://localhost:3000/api/labs
Запросов: 50, Одновременно: 10
```

Результаты:

```
Успешно (200): 0
Rate limited (429): 0
Ошибки: 50
Время: 0.01с
RPS: 7142.86
```

```
=====
ТЕСТ ЗАВЕРШЁН
=====
```

Интерпретация результатов:

- Если `rateLimited > 0`, `rate limiting` работает
- Если ошибок много, проверьте, запущен ли сервер
- RPS показывает пропускную способность

```
✓ test/loadTest.js (2130.81575ms)
✓ Недоверенный источник не получает разрешающий заголовок (0.731917ms)
✓ Доверенный источник получает правильные CORS заголовки (0.076167ms)
✓ Ограничитель частоты блокирует лишние запросы на чтение (0.1745ms)
✓ Ограничитель частоты разделяет чтение и запись (0.0785ms)
✓ Ограничитель возвращает корректную информацию об остатке (0.082667ms)
i tests 12
i suites 0
i pass 12
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 2142.398375
```

7.2. Проверка приоритета настроек

Тест **Аргументы командной строки имеют высший приоритет** подтверждает, что значение из CLI перекрывает и файл, и переменную окружения. Тест **Переменные окружения переопределяют файл** подтверждает средний уровень приоритета.

7.3. Проверка валидации

Запуск с заведомо некорректным значением:

```
node src/server.js --port=0
```

приводит к остановке запуска с подробным сообщением.

```
i duration_ms 2142.398375
❗ zhilovilya@zhilovilya-mac Courses_pr3 % node src/server.js --port=0
Запуск остановлен из-за некорректных настроек:
  - Порт задан неверно, значение должно быть целым числом от 1 до 65535
○ zhilovilya@zhilovilya-mac Courses_pr3 %
```

7.4. Проверка CORS

Запрос с недоверенным источником:

```
curl -i -H "Origin: http://evil.local" http://localhost:3000/api/labs
```

возвращает статус 403. В учебном режиме — с подробным сообщением, в боевом — **Forbidden**.

```
[zhilovilya@zhilovilya-mac ~ % curl -i -H "Origin: http://evil.local" http://localhost:3000/api/labs
HTTP/1.1 403 Forbidden
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Cache-Control: no-store, max-age=0
Pragma: no-cache
Content-Type: text/plain; charset=utf-8
Date: Sun, 10 May 2026 07:43:42 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Content-Length: 109
```

```
Доступ запрещён: источник "http://evil.local" не в списке доверенных%
zhilovilya@zhilovilya-mac ~ %
```

7.5. Проверка rate limiting

Скрипт `test/loadTest.js` поднимает по 30/100/50 запросов на маршруты чтения и записи. При высокой нагрузке часть запросов получает 429.

```
- RPS показывает пропускную способность
● zhilovilya@zhilovilya-mac Courses_pr3 % npm run test:load

> pr3-labs-config-and-security@1.0.0 test:load
> node test/loadTest.js

=====
      НАГРУЗОЧНОЕ ТЕСТИРОВАНИЕ
=====

Убедитесь, что сервер запущен (npm start)
Тест будет выполнен через 2 секунды...

Тест 1: Чтение, низкая нагрузка
  URL: http://localhost:3000/api/labs
  Запросов: 30, Одновременно: 5

Результаты:
  Успешно (200): 30
  Rate limited (429): 0
  Ошибки: 0
  Время: 0.05с
  RPS: 612.24

Тест 2: Чтение, высокая нагрузка
  URL: http://localhost:3000/api/labs
  Запросов: 100, Одновременно: 20

Результаты:
  Успешно (200): 30
  Rate limited (429): 70
```

7.6. Проверка защитных заголовков

```
curl -i http://localhost:3000/api/labs
```

В ответе присутствуют заголовки **X-Content-Type-Options: nosniff**, **X-Frame-Options: DENY**, **Cache-Control: no-store, max-age=0**.

```
● zhilovilya@zhilovilya-mac Courses_pr3 % curl -i http://localhost:3000/api/labs
HTTP/1.1 429 Too Many Requests
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Cache-Control: no-store, max-age=0
Pragma: no-cache
Content-Type: text/plain; charset=utf-8
Retry-After: 60
Date: Sun, 10 May 2026 07:45:54 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Content-Length: 152

Лимит запросов превышен. Доступно 60 запросов в минуту для чтения. Попробуйте позже.
○ zhilovilya@zhilovilya-mac Courses_pr3 %
```

8. Анализ критичных настроек

Настройка	Почему критична
<code>trustedOrigins</code>	Без неё любой сайт может вызывать API из браузера пользователя.
<code>rateLimits</code>	Защищает от DoS — один источник не может вытеснить остальных.
<code>mode</code>	В боевом режиме нельзя выдавать злоумышленнику подробности ошибок.
<code>port</code>	Неверный порт ломает всю доступность службы.

Раннее обнаружение ошибок конфигурации снижает риски: сервер никогда не запускается с опасной конфигурацией, проблемы видны на этапе деплоя, а не во время промышленной эксплуатации.

9. Выводы

В рамках практической работы №3 на Node.js разработана веб-служба учёта лабораторных работ. Полностью реализованы все требования задания:

1. Чтение настроек из трёх источников с явным приоритетом (файл → окружение → CLI).
2. Ранняя проверка корректности настроек с подробным выводом проблем.
3. Ограничение запросов из браузера только от доверенных источников (CORS, 403).
4. Раздельные лимиты на чтение и запись (60 и 20 запросов в минуту по умолчанию).
5. Защитные заголовки `X-Content-Type-Options`, `X-Frame-Options`, `Cache-Control`, `Pragma`.
6. Два режима работы — учебный и боевой — переключаемых только настройкой.

Дополнительно проведено модульное и нагрузочное тестирование, подтвердившее работоспособность всех заявленных механизмов.